

Les fichiers `graphes.mli` et `graphes.ml` contiennent respectivement l'interface d'une structure de graphes (orientés ou non orientés) et son implémentation.

Exercice 1 :**Composantes connexes, recherche de cycle**

Le fichier `ex01.ml` contient dans la définition de plusieurs graphes permettant de tester vos fonctions.

Écrire les fonctions suivantes, en utilisant un parcours en profondeur à chaque fois¹.

1. `est_connexe : graphe -> bool` qui prend en argument un graphe **non orienté** et indique s'il est connexe.
2. `composantes_connexes : graphe -> int array` qui prend en argument un graphe **non orienté** de taille n et renvoie un tableau `c` de n entiers tel que `c.(u) = c.(v)` si et seulement si u et v sont dans la même composante connexe, pour tous $u, v \in \llbracket 0; n-1 \rrbracket$.
3. `contient_cycle : graphe -> bool` qui prend en argument un graphe **orienté** et indique s'il contient un cycle.
4. `contient_cycle_non_oriente : graphe -> bool` qui prend en argument un graphe **non orienté** et indique s'il contient un cycle.

Exercice 2 :**SHAW !**

Le but de ce TP est en parti d'implémenter en C le tri topologique d'un graphe, mais également de travailler l'exploitation d'un fichier de données texte avec un programme.

On cherche à déterminer dans quel ordre on peut accomplir différentes tâches d'un jeu d'action-aventure (en l'occurrence, *Hollow Knight*). Le fichier `boss.txt` contient au début de chaque ligne le nom d'un des différents boss du jeu (et certains PNJ assimilés à des boss). Les termes suivants de la ligne, séparés par des espaces, indique la zone où se trouve le boss, et les prérequis éventuels pour l'affronter.

Ainsi, la ligne :

`Radiance Temple_of_the_Black_Egg Hollow_Knight Void_Heart Dream_Nail`

indique que pour battre "Radiance", il faut se trouver dans "Temple of the Black Egg", avoir vaincu "Hollow Knight", et être en possession du "Void Heart" et du "Dream Nail".

De même, `zones.txt` contient une liste des zones avec les prérequis² pour y accéder (autres zones à traverser, objets à récupérer au préalable, boss à battre) et `objets.txt` contient une liste similaire avec les objets³.

On rappelle les fonctions suivantes :

- `open_in : string -> in_channel` ouvre le fichier dont le chemin est fourni en argument (sous la forme d'une chaîne de caractère) et renvoie le flux entrant ainsi créé.

On le ferme avec `close_in : in_channel -> unit`.

- `input_line : in_channel -> string` prend en argument un flux entrant, et lit (consomme) tous les caractères jusqu'au prochain retour à la ligne (inclus). Il renvoie les caractères lus sous la forme d'une chaîne de caractères, sans le retour à la ligne final.
- `void fclose(FILE* stream)` libère la mémoire occupée par le flux `stream`.
- `String.split_on_char : char -> string -> string list` prend en argument un caractère séparateur `sep` et une chaîne de caractères `s`, et renvoie la liste des sous-chaînes (éventuellement vides) de `s` délimitées par `sep` ou par les extrémités de `s`.

1. Sans regarder la page suivante, comment proposez-vous d'implémenter le graphe des boss, objets et zones, dont les arcs sont les prérequis, afin d'en calculer un tri topologique ?

1. Les deux premières pourraient aussi se faire avec un parcours en largeur

2. Plusieurs simplifications ont été faites ici, notamment lorsqu'il existait un moyen détourné complexe d'accéder à une zone, il peut avoir été interdit par cette liste.

3. Encore une fois, quelques simplifications ont été faites. De plus, la liste est loin d'être exhaustive, mais contient au moins tous les objets qui sont les pré-requis d'un autre objet, d'une zone ou d'un boss.

2. Écrire une fonction `maj_sommets` qui prend en argument un chemin `nomDeFichier : string`, deux tables de hachage `numeros : (string, int) Hashtbl.t` et `noms : (int, string) Hashtbl.t`, un entier n tel que `numeros` et `noms` contient n associations.

La fonction parcourt le fichier dont le chemin est en premier argument, attribue à chaque nom rencontré en début de ligne un nouveau numéro i en partant de n , et stocke l'association entre ce nom et ce numéro :

- dans le tableau associatif `numeros`, de façon à ce que le nom soit une clé de `numeros`, associée à la valeur i ;
- dans le tableau associatif `noms`, de façon à ce que la valeur de clé i de `noms` soit le nom lu ;

Cette fonction renvoie le premier entier supérieur à n non attribué à une ligne du fichier.

3. Écrire une fonction `maj_graph : string -> (string, int) Hashtbl.t -> graph -> unit` qui, étant donné un `nomDeFichier` le fichier à lire, `numeros` un tableau associatif associant un entier de 0 à $n - 1$ à chacun des noms qui peuvent être rencontrés dans `fichier`, et G un graphe sur les sommets $[0; n - 1]$, ajoute à G tous les arcs correspondant à un prérequis donné dans le fichier `fichier`.
4. Écrire une fonction `tri_topologique graph -> int list` qui calcule un tri topologique d'un graphe G orienté acyclique, sous forme de liste chaînée.
5. Afficher un ordre possible pour battre tous les boss du jeu (ne pas afficher les objets ni les zones).
6. On souhaite maintenant s'intéresser à des parcours qui ne vont que jusqu'à battre un certain boss et ne cherchent pas à parcourir tout le jeu.
- (a) Que faut-il changer pour avoir un tri topologique qui porte uniquement sur les prérequis d'un boss donné ?
 - (b) Afficher un ordre possible pour battre uniquement les boss nécessaires afin d'obtenir la première fin du jeu : battre le boss "Hollow Knight".
 - (c) Afficher la liste des boss qu'il n'est pas nécessaire de vaincre pour battre le boss final : "Radiance".